Universidad de Costa Rica

Escuela de Ciencias de la Computación e Informática

CI-0141 | G01

BASES DE DATOS AVANZADAS

Laboratorio 7

Bases de Datos NoSQL - Orientadas a Documentos

Profesor:

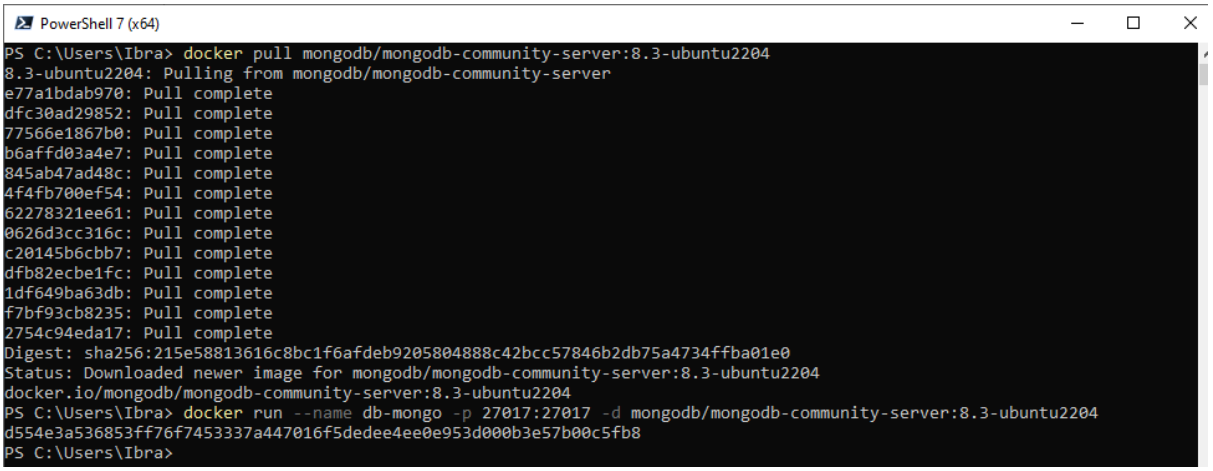
Mag. Luis Carlos Salas

Javier Cruz Ruiz C02517

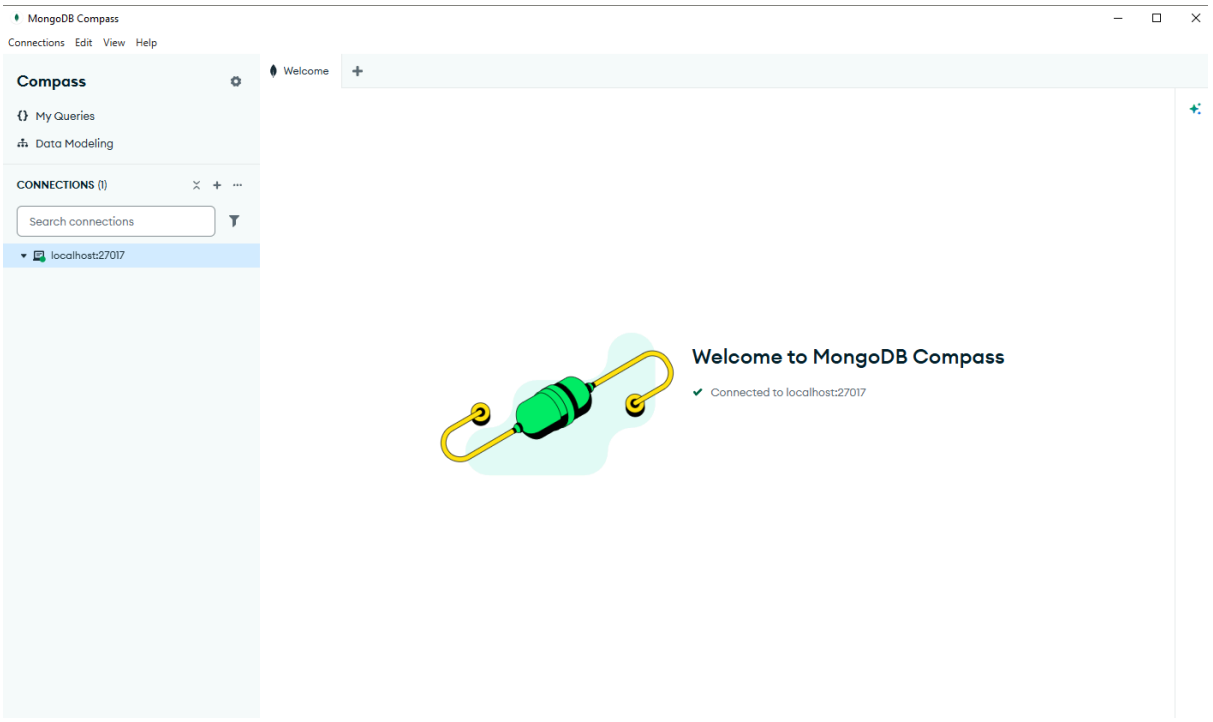
12 de Noviembre, 2026

Creación de una instancia standalone

1. Realice los siguientes pasos para realizar la habilitación de una instancia standalone de Mongo, utilizando un sólo "servidor", simulado utilizando un contenedor en Docker.

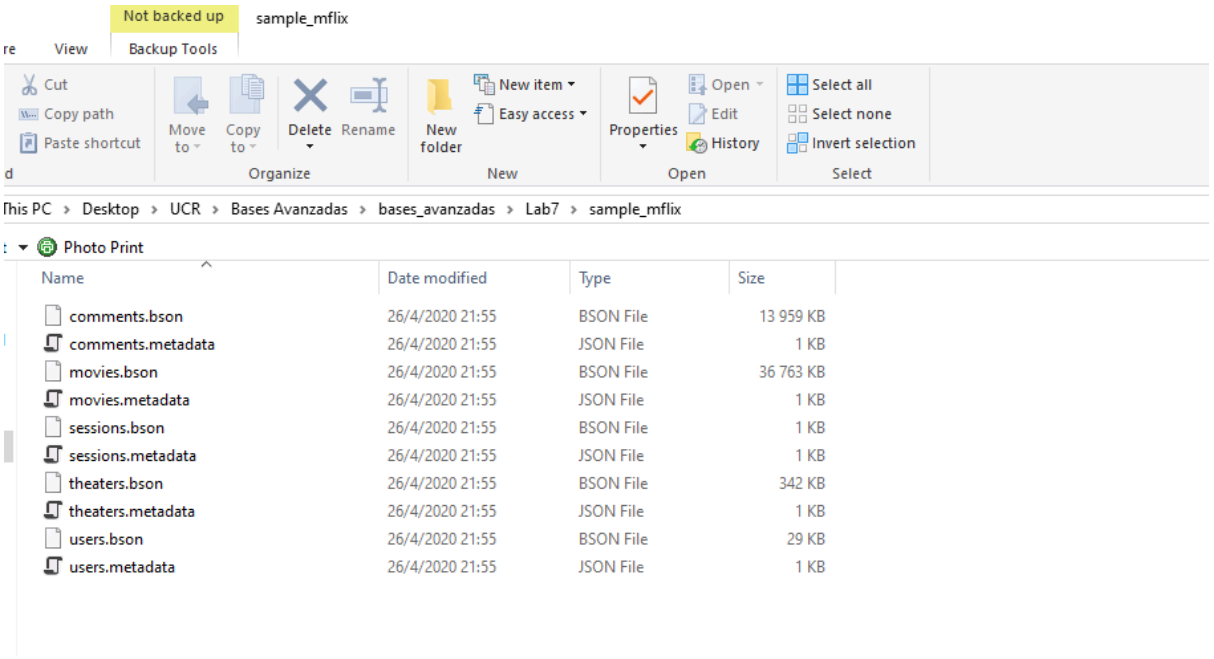


2. Establezca una conexión la instancia standalone de Mongo usando el Mongo Compass, utilizando la siguiente cadena de conexión: mongodb://localhost:27017/

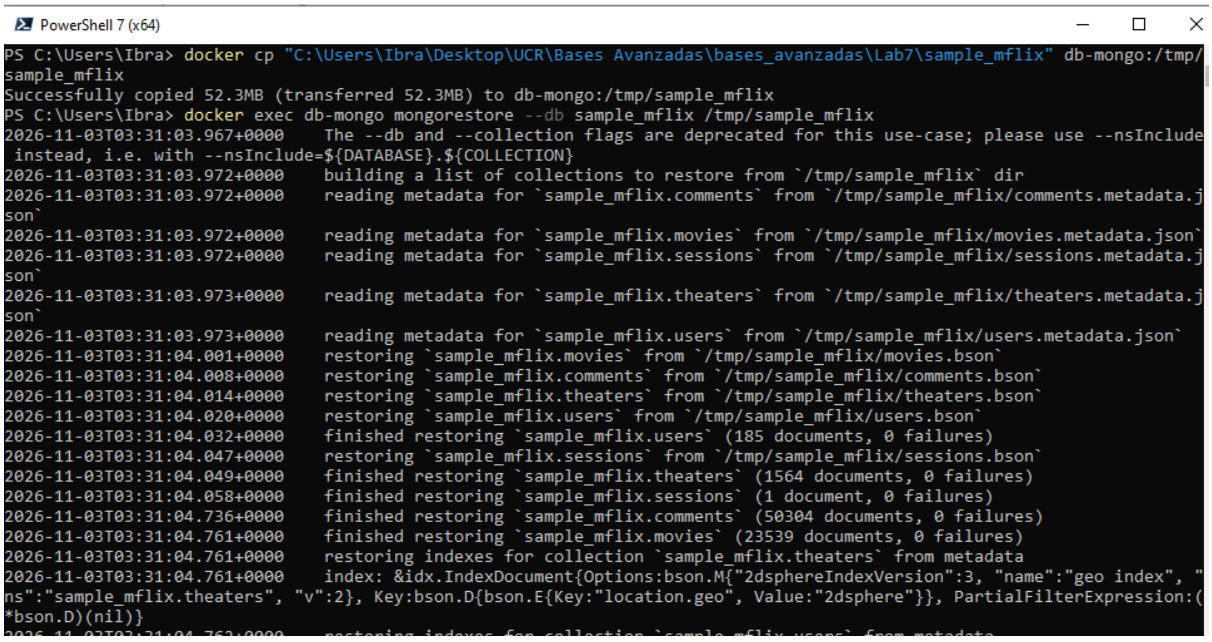


Restauración de base de datos

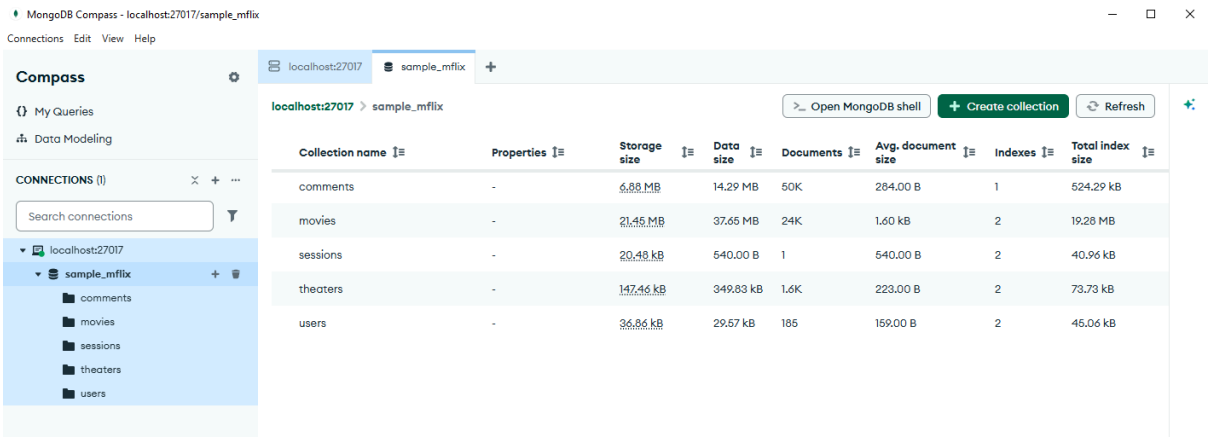
1. Descargue desde el repositorio la carpeta \dump\sample_mflix y ubíquela en una carpeta en su máquina local, por ejemplo, en “C:\mongo\sample_mflix”.



2. Ejecute los siguientes comandos en el contenedor "db-mongo" para restaurar la base de datos "sample_mflix" usando los archivos en /tmp/sample_mflix. El detalle de los comandos puede verlos en el material del curso.

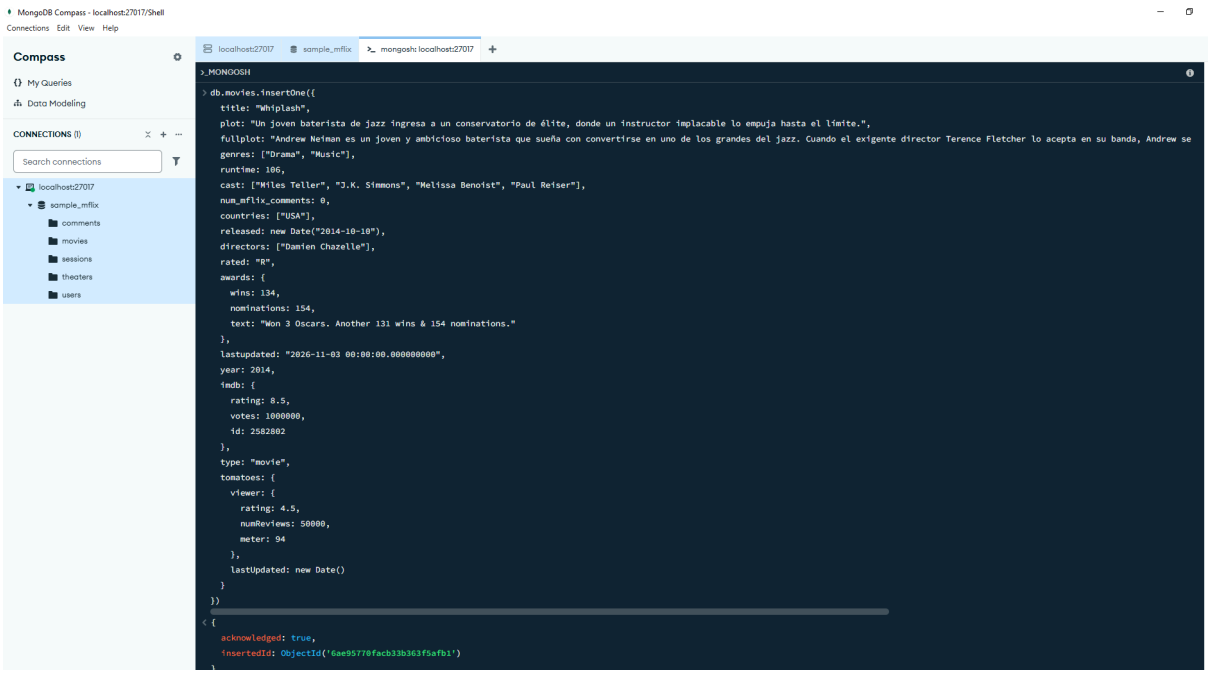


3. Establezca una conexión la instancia standalone de Mongo usando el Mongo Compass, utilizando la siguiente cadena de conexión:



Gestión de datos

1. Inserte en la colección movies su película favorita. El documento debe respetar una estructura similar a la utilizada por la colección.



2. Inserte en la colección comments dos comentarios sobre su película favorita. El documento debe respetar una estructura similar a la utilizada por la colección.

```
> const peli = db.movies.findOne({ title: "Whiplash" })
> db.comments.insertMany([
    {
      name: "Javier",
      email: "javier.cruzruiz@ucr.ac.cr",
      movie_id: peli._id,
      text: "Una película increíble, la banda sonora es espectacular.",
      date: new Date()
    },
    {
      name: "Javier",
      email: "javier.cruzruiz@ucr.ac.cr",
      movie_id: peli._id,
      text: "Mi pelicula favorita.",
      date: new Date()
    }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6ae95879facb33b363f5afb2'),
      '1': ObjectId('6ae95879facb33b363f5afb3')
    }
  }
sample_mflix>
```

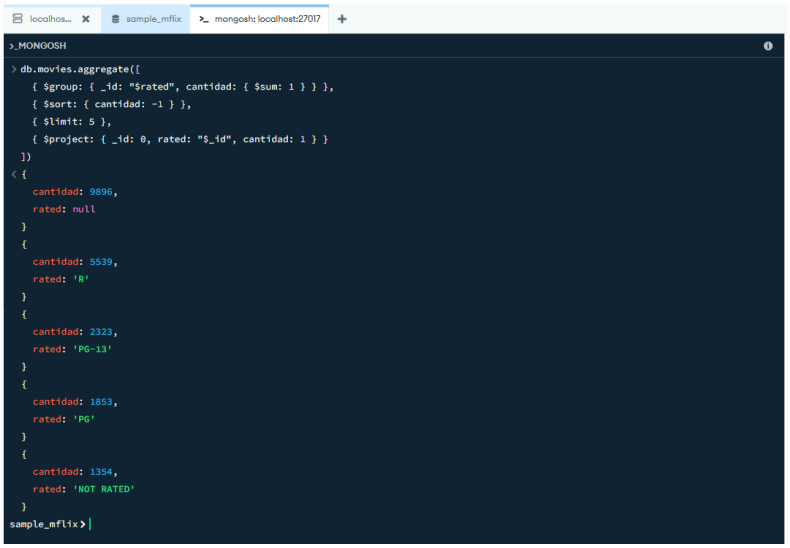
3. Actualice en su película favorita los campos plot y rating del imdb. Recupere la para verificar que los cambios fueron aplicados correctamente.

```
localhost:27017   sample_mflix   mongosh: localhost:27017   +
>_MONGOSH
> db.movies.updateOne(
    { title: "Whiplash" },
    {
      $set: {
        plot: "Nuevo resumen de la trama.",
        "imdb.rating": 9.0
      }
    }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
> db.movies.findOne(
    { title: "Whiplash" },
    { title: 1, plot: 1, "imdb.rating": 1 }
  )
< {
    _id: ObjectId('573a13dff29313caabdb85fb'),
    imdb: {
      rating: 9
    },
    plot: 'Nuevo resumen de la trama.',
    title: 'Whiplash'
  }
sample_mflix>
```

4. Implemente una consulta utilizando un cursor sobre la colección movies que recupere el título (title) de las 10 películas del género (genres) "Drama" ordenadas por fecha de estreno (released), de más reciente a más antigua.



5. Implemente una consulta utilizando el framework de agregación sobre la colección movies que obtenga, para cada clasificación (rated), la cantidad de películas registradas. El resultado debe mostrar solamente la clasificación (rated) y la cantidad, ordenado de mayor a menor según la cantidad, mostrando únicamente las primeras 5 clasificaciones.



6. Implemente un índice para optimizar la siguiente consulta sobre la colección comments. La consulta recupera los comentarios realizados por un usuario específico, mostrando únicamente el nombre, el correo, el texto y la fecha de los comentarios, ordenados del más reciente a más antiguo. Inclua el código del índice y tome una captura de pantalla del plan de ejecución.

```
>_MONGOSH
> db.comments.find(
    { email: "andrea_le@fakegmail.com" },
    { _id: 0, name: 1, email: 1, text: 1, date: 1 }
  ).sort({ date: -1 }).explain("executionStats")
< {
    explainVersion: '1',
    queryPlanner: {
      namespace: 'sample_mflix.comments',
      parsedQuery: {
        email: {
          '$eq': 'andrea_le@fakegmail.com'
        }
      },
      indexFilterSet: false,
      queryHash: '2581EED4',
      planCacheShapeHash: '2581EED4',
      planCacheKey: '5BDEA732',
      optimizationTimeMillis: 2,
      maxIndexedOrSolutionsReached: false,
      maxIndexedAndSolutionsReached: false,
      maxScansToExplodeReached: false,
      prunedSimilarIndexes: false,
      winningPlan: {
        isCached: false,
        stage: 'SORT',
        sortPattern: {
          date: -1
        },
        memLimit: 104857600,
        type: 'simple',
        inputStage: {
          stage: 'PROJECTION_SIMPLE',
          transformBy: {
            _id: 0,
            name: 1,
            email: 1,
```

```
>_MONGOSH
            date: 1
          },
          inputStage: {
            stage: 'COLLSCAN',
            filter: {
              email: {
                '$eq': 'andrea_le@fakegmail.com'
              }
            },
            nss: 'sample_mflix.comments',
            direction: 'forward'
          }
        }
      },
      rejectedPlans: []
    },
    executionStats: {
      executionSuccess: true,
      nReturned: 296,
      executionTimeMillis: 34,
      totalKeysExamined: 0,
      totalDocsExamined: 50306,
      executionStages: {
        isCached: false,
        stage: 'SORT',
        nReturned: 296,
        executionTimeMillisEstimate: 10,
        works: 50604,
        advanced: 296,
        needTime: 50307,
        needYield: 0,
        saveState: 1,
        restoreState: 1,
        isEOF: 1,
        sortPattern: {
          date: -1
        },
```

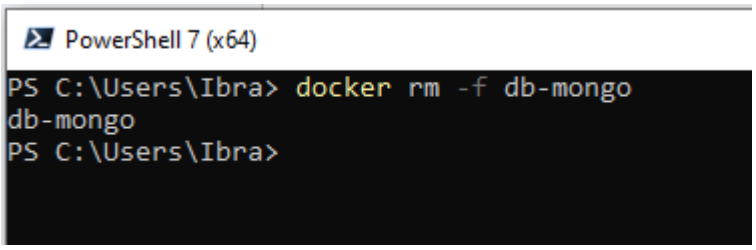
```
> db.comments.createIndex({ email: 1, date: -1 })
< email_1_date_-1
sample_mflix>
```

```
> db.comments.find(
      { email: "andrea_le@fakegmail.com" },
      { _id: 0, name: 1, email: 1, text: 1, date: 1 }
  ).sort({ date: -1 }).explain("executionStats")
< {
    explainVersion: '1',
    queryPlanner: {
      namespace: 'sample_mflix.comments',
      parsedQuery: {
        email: {
          '$eq': 'andrea_le@fakegmail.com'
        }
      },
      indexFilterSet: false,
      queryHash: '2581EED4',
      planCacheShapeHash: '2581EED4',
      planCacheKey: '070106C5',
      optimizationTimeMillis: 1,
      maxIndexedOrSolutionsReached: false,
      maxIndexedAndSolutionsReached: false,
      maxScansToExplodeReached: false,
      prunedSimilarIndexes: false,
      winningPlan: {
        isCached: false,
        stage: 'PROJECTION_SIMPLE',
        transformBy: {
          _id: 0,
          name: 1,
          email: 1,
          text: 1,
          date: 1
        },
        inputStage: {
          stage: 'FETCH',
          nss: 'sample_mflix.comments',
          inputStage: {
            stage: 'IXSCAN',
```


localhost:27017
sample_mflix
mongosh: localhost:27017
>_MONGOSH
nss: 'sample_mflix.comments',
keyPattern: {
email: 1,
date: -1
},
indexName: 'email_1_date_-1',
isMultiKey: false,
multiKeyPaths: {
email: [],
date: []
},
isUnique: false,
isSparse: false,
isPartial: false,
indexVersion: 2,
direction: 'forward',
indexBounds: {
email: [
'["andrea_le@fakegmail.com", "andrea_le@fakegmail.com"]'
],
date: [
'[MaxKey, MinKey]'
]
}
}
}
},
rejectedPlans: []
},
executionStats: {
executionSuccess: true,
nReturned: 296,
executionTimeMillis: 3,
totalKeysExamined: 296,
totalDocsExamined: 296,
executionStages: {
isCached: false,
stage: 'PROJECTION_SIMPLE',
nReturned: 296.

Eliminación de la instancia standalone

1. Ejecute los siguientes comandos para eliminar el contenedor y los volúmenes de la instancia standalone de Mongo. Asegúrese de haber terminado esta parte del laboratorio, ya que la eliminación no se puede deshacer.

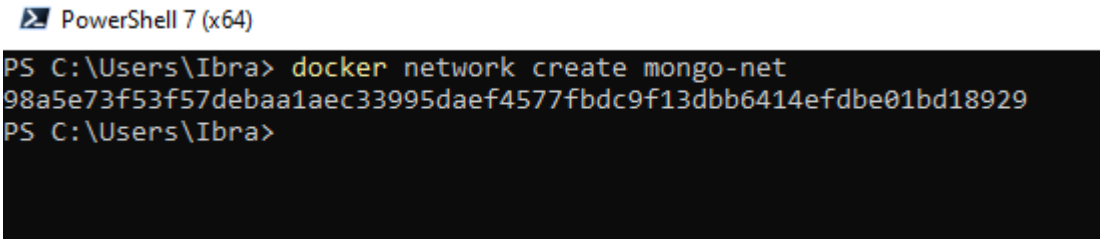


PARTE 2

Creación de un replica set

1. Realice los siguientes pasos para realizar la habilitación de un replica set de Mongo, utilizando tres "servidores", simulados utilizando tres contenedores en Docker.

a. Ejecute el siguiente comando para crear una red interna para comunicar los contenedores entre sí.



b. Ejecute los siguientes comandos para iniciar un replica set de tres nodos de Mongo, donde cada nodo se desplegará en un contenedor independiente. El nodo 1 utilizará el puerto 27018, el nodo 2 utilizará el puerto 27019 y el nodo 3 utilizará el puerto 27020. El puerto específico de cada nodo permitirá hacer una conexión directa con cada contenedor correspondiente. La comunicación interna entre nodos del replicase set será por lared interna.

```
PowerShell 7 (x64)
PS C:\Users\Ibra> docker run --name mongo-node-1 --network mongo-net -p 27018:27017 -d mongodb/mongodb-community-server:
8.3-ubuntu2204 --replSet rs0 --bind_ip_all
c81ea04b686ea6701b4a277617f9ab56cb4e651814e274e35adabf7f763effeb
PS C:\Users\Ibra> docker run --name mongo-node-2 --network mongo-net -p 27019:27017 -d mongodb/mongodb-community-server:
8.3-ubuntu2204 --replSet rs0 --bind_ip_all
b6ecda225a9e19619e2ffa80a7d6362729461d2d2c505a8083f139a68ffd21a8
PS C:\Users\Ibra> docker run --name mongo-node-3 --network mongo-net -p 27020:27017 -d mongodb/mongodb-community-server:
8.3-ubuntu2204 --replSet rs0 --bind_ip_all
8c828f2048e541241cccbe142bc3c4ef1d52a5104b3fa47c12f66fece814e63d
PS C:\Users\Ibra>
```

c. Ejecute el siguiente comando para establecer la conexión con el nodo 1 usando el mongosh.

```
mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
PS C:\Users\Ibra> docker exec -it mongo-node-1 mongosh
Current Mongosh Log ID: 6ae95cbf9ee269a3738ce5af
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2
.6.0
Using MongoDB:          8.3.13
Using Mongosh:          2.6.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.co
m/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

------
   The server generated these startup warnings when booting
   2026-11-03T04:01:10.937+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. S
ee http://dochub.mongodb.org/core/prodnotes-filesystem
   2026-11-03T04:01:11.096+00:00: Access control is not enabled for the database. Read and write access to data and conf
iguration is unrestricted
   2026-11-03T04:01:11.096+00:00: For customers running the current memory allocator, we suggest changing the contents o
f the following sysfsFile
   2026-11-03T04:01:11.096+00:00: For customers running the current memory allocator, we suggest changing the contents o
f the following sysfsFile
   2026-11-03T04:01:11.096+00:00: We suggest setting the contents of sysfsFile to 0.
   2026-11-03T04:01:11.096+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
------

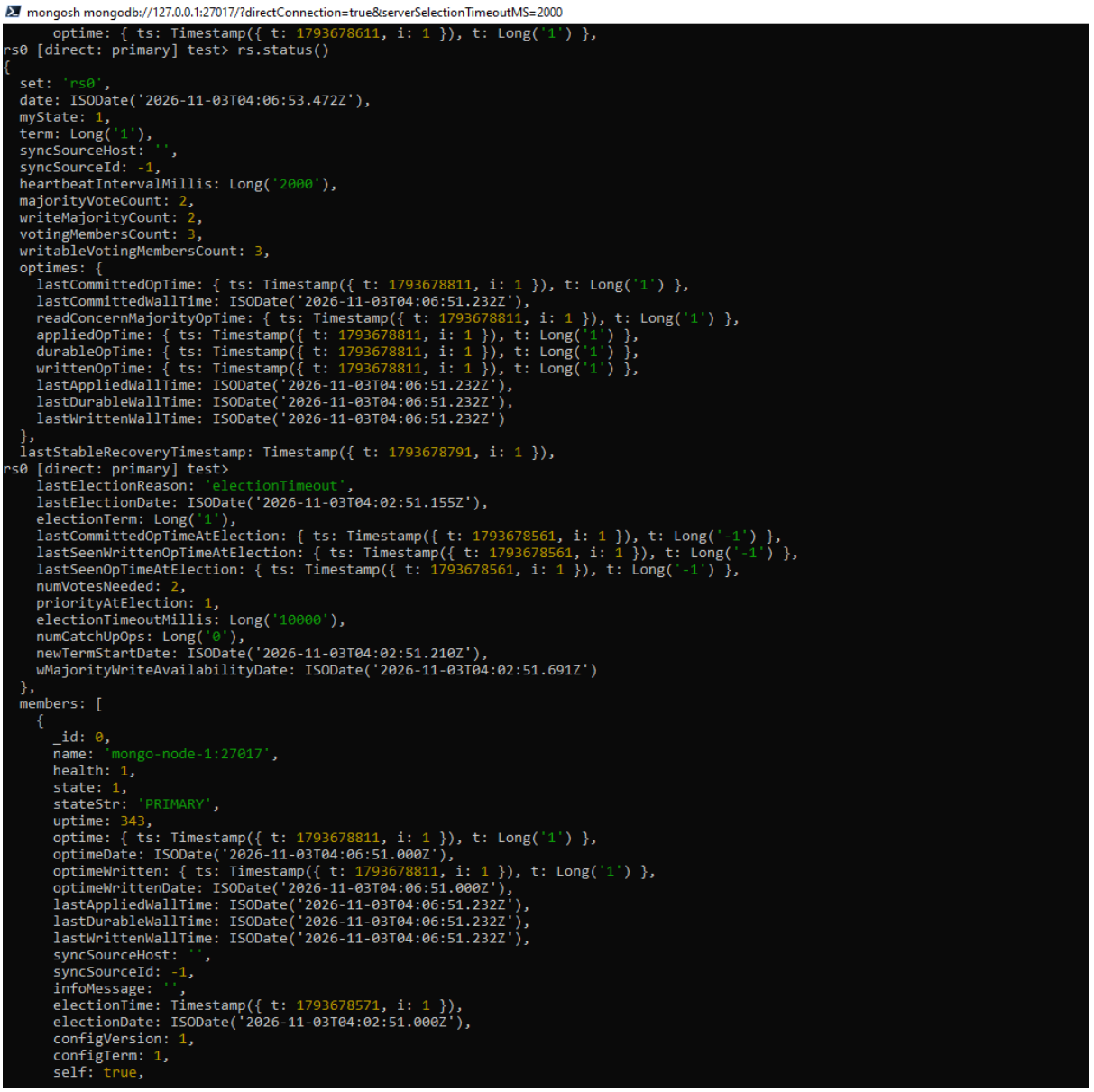
test>
```

d. Ejecute el siguiente comando en el nodo 1 usando el mongosh para inicializar el replica set con una configuración de miembros.

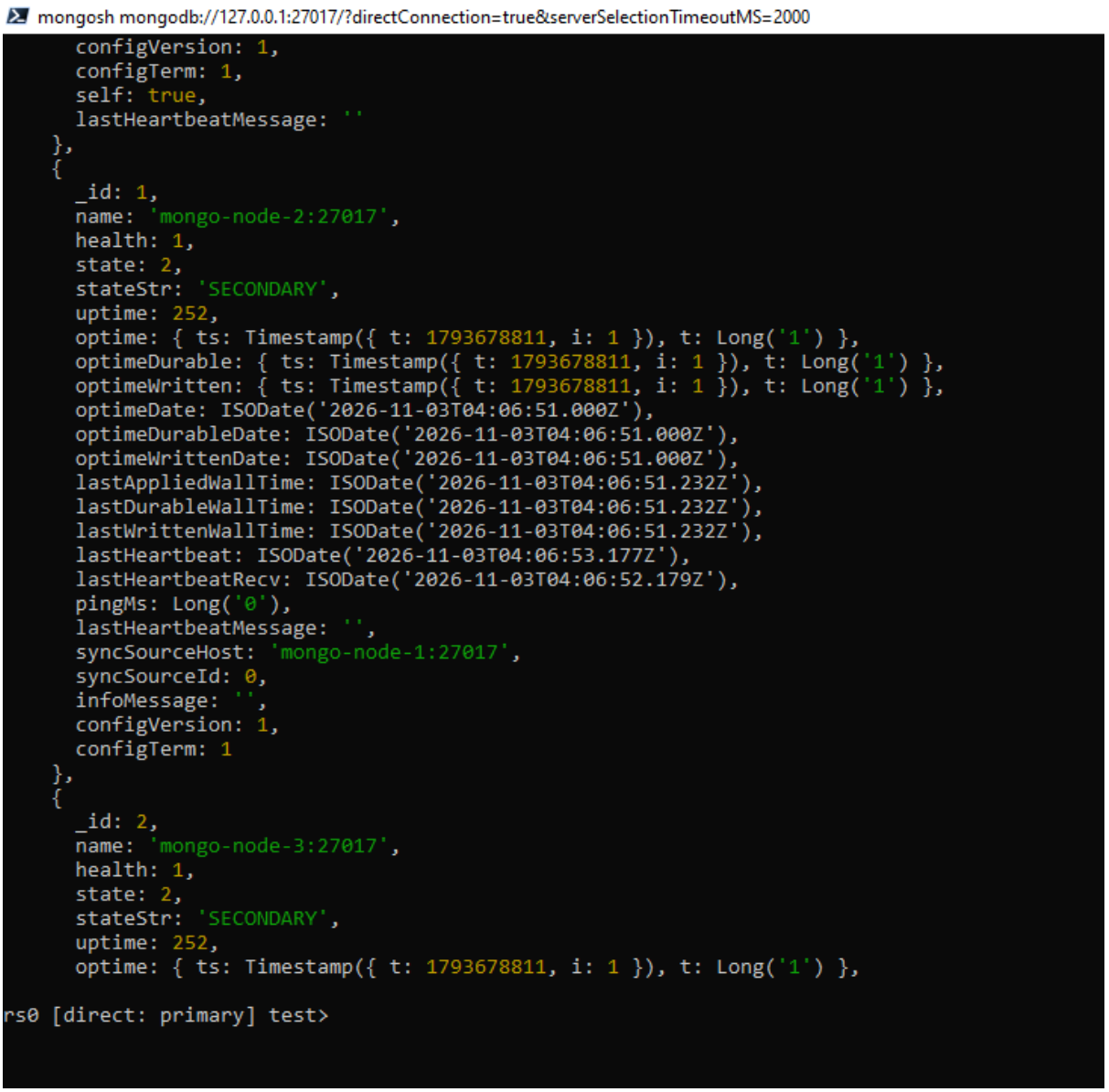
```
mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
   2026-11-03T04:01:11.096+00:00: Access control is not enabled for the database. Read and write access to data and conf
iguration is unrestricted
   2026-11-03T04:01:11.096+00:00: For customers running the current memory allocator, we suggest changing the contents o
f the following sysfsFile
   2026-11-03T04:01:11.096+00:00: For customers running the current memory allocator, we suggest changing the contents o
f the following sysfsFile
   2026-11-03T04:01:11.096+00:00: We suggest setting the contents of sysfsFile to 0.
   2026-11-03T04:01:11.096+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
------

test> rs.initiate({
...   _id: "rs0",
...   members: [
...     { _id: 0, host: "mongo-node-1:27017" },
...     { _id: 1, host: "mongo-node-2:27017" },
...     { _id: 2, host: "mongo-node-3:27017" }
...   ]
... })
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793678561, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793678561, i: 1 })
}
rs0 [direct: secondary] test>
```

e. Ejecute el siguiente comando en el nodo 1 usando el mongosh para verificar el status del replica set.



```
mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
    optime: { ts: Timestamp({ t: 1793678611, i: 1 }), t: Long('1') },
rs0 [direct: primary] test> rs.status()
{
  set: 'rs0',
  date: ISODate('2026-11-03T04:06:53.472Z'),
  myState: 1,
  term: Long('1'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1793678811, i: 1 }), t: Long('1') },
    lastCommittedWallTime: ISODate('2026-11-03T04:06:51.232Z'),
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1793678811, i: 1 }), t: Long('1') },
    appliedOpTime: { ts: Timestamp({ t: 1793678811, i: 1 }), t: Long('1') },
    durableOpTime: { ts: Timestamp({ t: 1793678811, i: 1 }), t: Long('1') },
    writtenOpTime: { ts: Timestamp({ t: 1793678811, i: 1 }), t: Long('1') },
    lastAppliedWallTime: ISODate('2026-11-03T04:06:51.232Z'),
    lastDurableWallTime: ISODate('2026-11-03T04:06:51.232Z'),
    lastWrittenWallTime: ISODate('2026-11-03T04:06:51.232Z')
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1793678791, i: 1 }),
rs0 [direct: primary] test>
    lastElectionReason: 'electionTimeout',
    lastElectionDate: ISODate('2026-11-03T04:02:51.155Z'),
    electionTerm: Long('1'),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1793678561, i: 1 }), t: Long('-1') },
    lastSeenWrittenOpTimeAtElection: { ts: Timestamp({ t: 1793678561, i: 1 }), t: Long('-1') },
    lastSeenOpTimeAtElection: { ts: Timestamp({ t: 1793678561, i: 1 }), t: Long('-1') },
    numVotesNeeded: 2,
    priorityAtElection: 1,
    electionTimeoutMillis: Long('10000'),
    numCatchUpOps: Long('0'),
    newTermStartDate: ISODate('2026-11-03T04:02:51.210Z'),
    wMajorityWriteAvailabilityDate: ISODate('2026-11-03T04:02:51.691Z')
  },
  members: [
    {
      _id: 0,
      name: 'mongo-node-1:27017',
      health: 1,
      state: 1,
      stateStr: 'PRIMARY',
      uptime: 343,
      optime: { ts: Timestamp({ t: 1793678811, i: 1 }), t: Long('1') },
      optimeDate: ISODate('2026-11-03T04:06:51.000Z'),
      optimeWritten: { ts: Timestamp({ t: 1793678811, i: 1 }), t: Long('1') },
      optimeWrittenDate: ISODate('2026-11-03T04:06:51.000Z'),
      lastAppliedWallTime: ISODate('2026-11-03T04:06:51.232Z'),
      lastDurableWallTime: ISODate('2026-11-03T04:06:51.232Z'),
      lastWrittenWallTime: ISODate('2026-11-03T04:06:51.232Z'),
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: '',
      electionTime: Timestamp({ t: 1793678571, i: 1 }),
      electionDate: ISODate('2026-11-03T04:02:51.000Z'),
      configVersion: 1,
      configTerm: 1,
      self: true,
```



2. Conteste las siguientes preguntas:

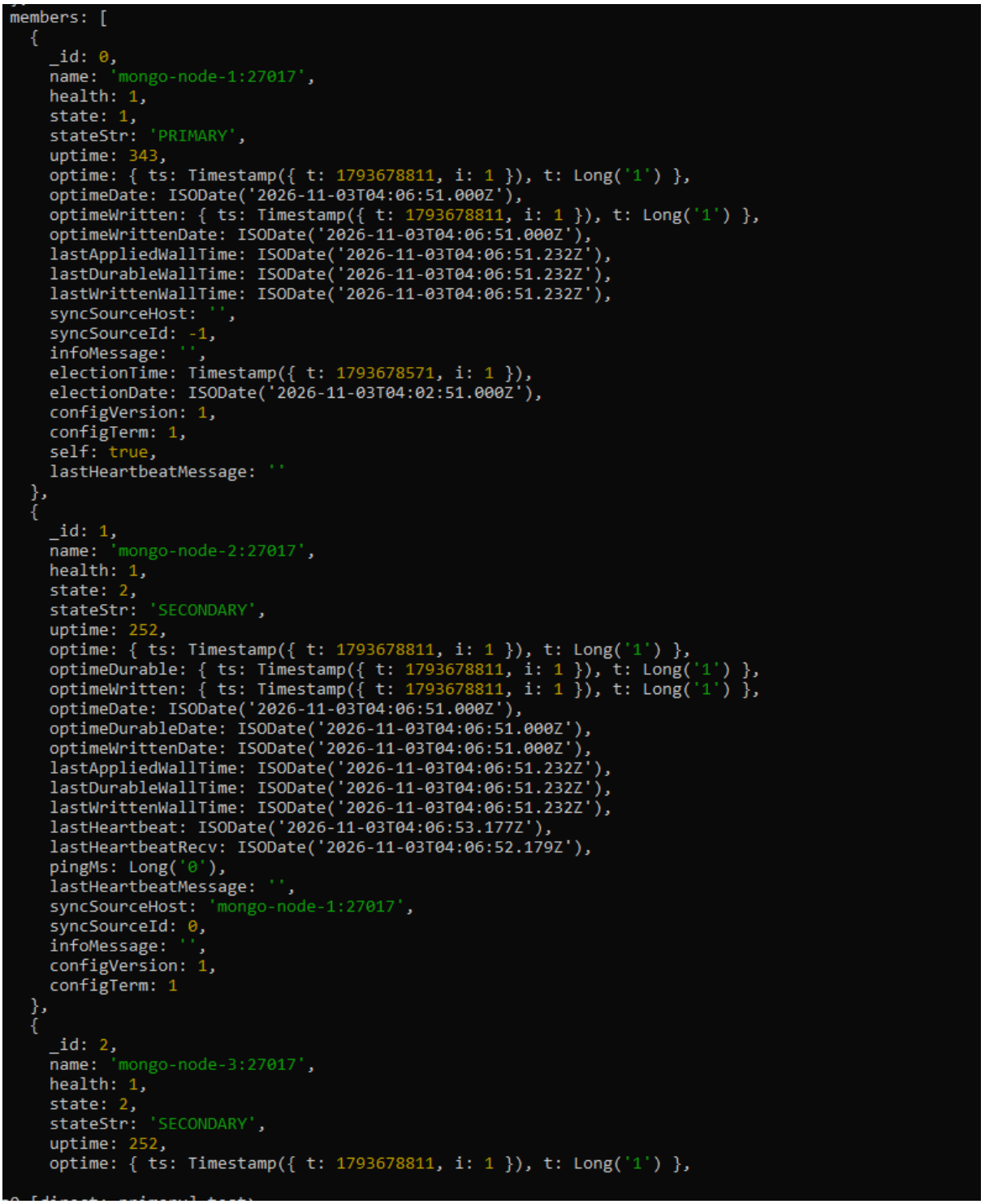
a. ¿Cuál nodo está funcionando como PRIMARY?

'mongo-node-1:27017', el nodo 1 está funcionando como Primary.

b. ¿Cuáles nodos están funcionando como SECONDARY?

'mongo-node-2:27017' y 'mongo-node-3:27017', el nodo 2 y 3 están funcionando como Secondary.

c. Tome una captura de pantalla donde se pueda identificar el nodo primario (“PRIMARY”) y los nodos secundarios (“SECONDARY”).



```
members: [
  {
    _id: 0,
    name: 'mongo-node-1:27017',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 343,
    optime: { ts: Timestamp({ t: 1793678811, i: 1 }), t: Long('1') },
    optimeDate: ISODate('2026-11-03T04:06:51.000Z'),
    optimeWritten: { ts: Timestamp({ t: 1793678811, i: 1 }), t: Long('1') },
    optimeWrittenDate: ISODate('2026-11-03T04:06:51.000Z'),
    lastAppliedWallTime: ISODate('2026-11-03T04:06:51.232Z'),
    lastDurableWallTime: ISODate('2026-11-03T04:06:51.232Z'),
    lastWrittenWallTime: ISODate('2026-11-03T04:06:51.232Z'),
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1793678571, i: 1 }),
    electionDate: ISODate('2026-11-03T04:02:51.000Z'),
    configVersion: 1,
    configTerm: 1,
    self: true,
    lastHeartbeatMessage: ''
  },
  {
    _id: 1,
    name: 'mongo-node-2:27017',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 252,
    optime: { ts: Timestamp({ t: 1793678811, i: 1 }), t: Long('1') },
    optimeDurable: { ts: Timestamp({ t: 1793678811, i: 1 }), t: Long('1') },
    optimeWritten: { ts: Timestamp({ t: 1793678811, i: 1 }), t: Long('1') },
    optimeDate: ISODate('2026-11-03T04:06:51.000Z'),
    optimeDurableDate: ISODate('2026-11-03T04:06:51.000Z'),
    optimeWrittenDate: ISODate('2026-11-03T04:06:51.000Z'),
    lastAppliedWallTime: ISODate('2026-11-03T04:06:51.232Z'),
    lastDurableWallTime: ISODate('2026-11-03T04:06:51.232Z'),
    lastWrittenWallTime: ISODate('2026-11-03T04:06:51.232Z'),
    lastHeartbeat: ISODate('2026-11-03T04:06:53.177Z'),
    lastHeartbeatRecv: ISODate('2026-11-03T04:06:52.179Z'),
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
    syncSourceHost: 'mongo-node-1:27017',
    syncSourceId: 0,
    infoMessage: '',
    configVersion: 1,
    configTerm: 1
  },
  {
    _id: 2,
    name: 'mongo-node-3:27017',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 252,
    optime: { ts: Timestamp({ t: 1793678811, i: 1 }), t: Long('1') },
```

Funcionamiento del replica set

1. Realice los siguientes pasos para crear una base de datos y una colección en el replica set.

a. Establezca una conexión con el nodo primario usando el mongosh.

```
mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
rs0 [direct: primary] test>
```

b. Ejecute los siguientes comandos para la creación de la base de datos y la colección.

```
mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
rs0 [direct: primary] test> use tienda
switched to db tienda
rs0 [direct: primary] tienda>

rs0 [direct: primary] tienda> db.productos.insertMany([
...     {
...         codigo: "PROD-001",
...         nombre: "Libro",
...         precio: 25000,
...         existencias: 5
...     },
...     {
...         codigo: "PROD-002",
...         nombre: "Cuaderno ",
...         precio: 3500,
...         existencias: 25
...     },
...     {
...         codigo: "PROD-003",
...         nombre: "Lapiz",
...         precio: 500,
...         existencias: 15
...     }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ae9659d9ee269a3738ce5b0'),
    '1': ObjectId('6ae9659d9ee269a3738ce5b1'),
    '2': ObjectId('6ae9659d9ee269a3738ce5b2')
  }
}
rs0 [direct: primary] tienda>

rs0 [direct: primary] tienda> db.productos.find()
```

2. Conteste las siguientes preguntas:

a. ¿Por qué la escritura debe ejecutarse sobre el nodo primario?

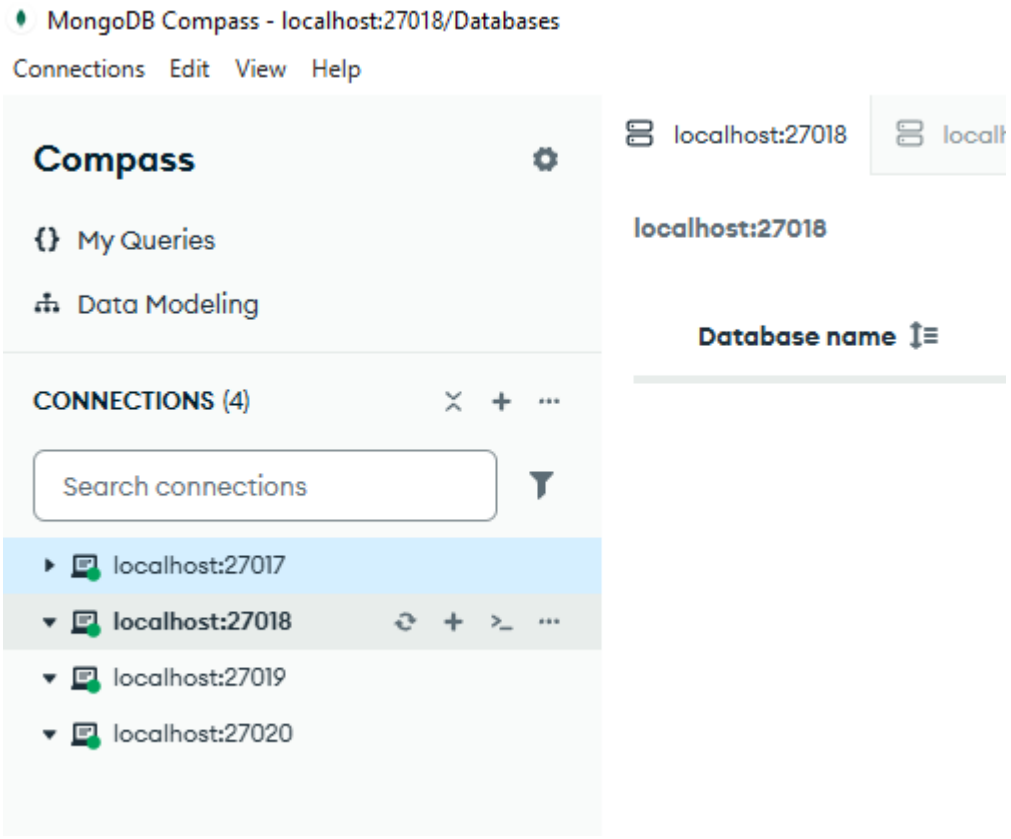
Debe ejecutarse sobre el nodo primario ya que en un set, solo el primario acepta escrituras. Posteriormente se registra cada operación en su log, y los secundarios lo aplican usando ese log. Si se trata de escribir sobre un secundario, se rechaza la operación.

b. Tome una captura de pantalla donde se observen los documentos productos y se vea su identificador _id.

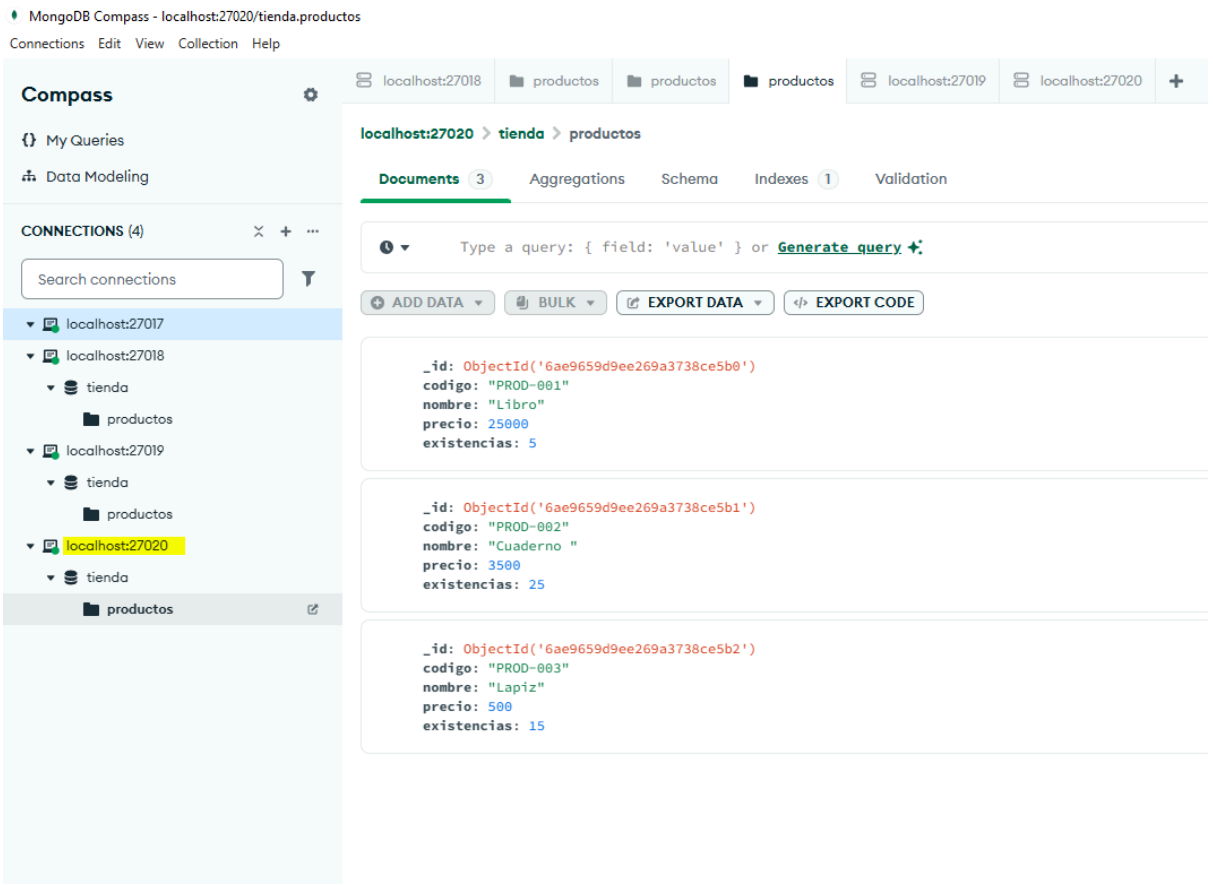
```
rs0 [direct: primary] tienda> db.productos.find()
[
  {
    _id: ObjectId('6ae9659d9ee269a3738ce5b0'),
    codigo: 'PROD-001',
    nombre: 'Libro',
    precio: 25000,
    existencias: 5
  },
  {
    _id: ObjectId('6ae9659d9ee269a3738ce5b1'),
    codigo: 'PROD-002',
    nombre: 'Cuaderno ',
    precio: 3500,
    existencias: 25
  },
  {
    _id: ObjectId('6ae9659d9ee269a3738ce5b2'),
    codigo: 'PROD-003',
    nombre: 'Lapiz',
    precio: 500,
    existencias: 15
  }
]
rs0 [direct: primary] tienda>
```

3. Realice los siguientes pasos para verificar la replicación de la base de datos y la colección en los nodos secundarios del replica set.

a. Establezca una conexión con un de los nodos secundarios usando el Mongo Compass, utilizando sólo una de las siguientes cadenas de conexión según corresponda:



b. Explore las bases de datos y las colecciones existentes.



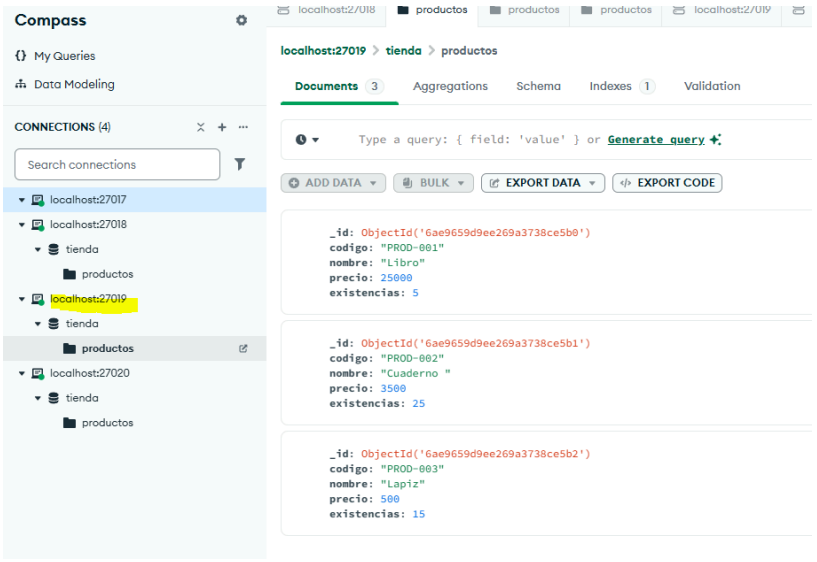
4. Conteste las siguientes preguntas:

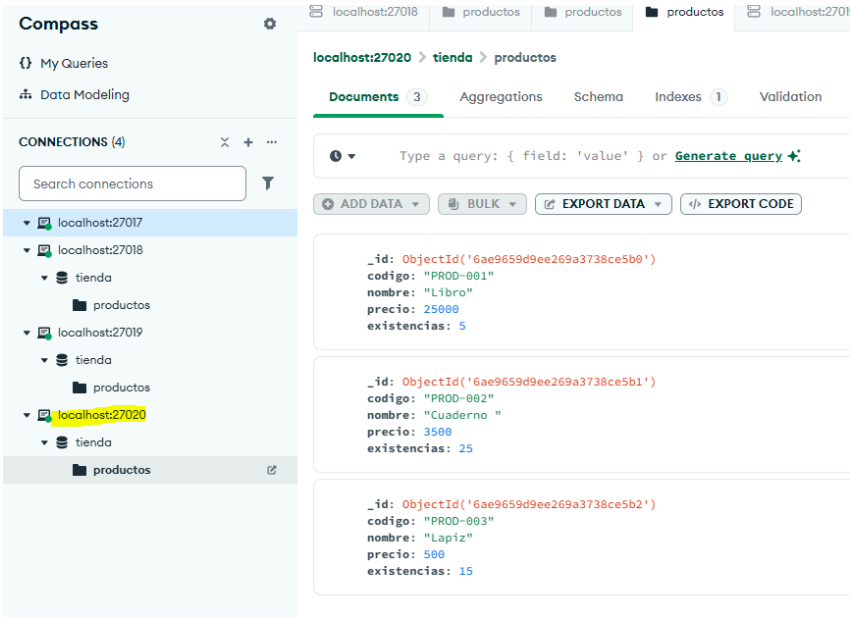
a. ¿Los documentos insertados en el nodo primario aparece también en el nodo secundario?

Sí, en los nodos secundarios también aparecen los documentos insertados en el nodo primario.

b. Tome una captura de pantalla donde se observen los documentos productos y se vea su identificador _id.

```
rs0 [direct: primary] tienda> db.productos.find()
[
  {
    _id: ObjectId('6ae9659d9ee269a3738ce5b0'),
    codigo: 'PROD-001',
    nombre: 'Libro',
    precio: 25000,
    existencias: 5
  },
  {
    _id: ObjectId('6ae9659d9ee269a3738ce5b1'),
    codigo: 'PROD-002',
    nombre: 'Cuaderno ',
    precio: 3500,
    existencias: 25
  },
  {
    _id: ObjectId('6ae9659d9ee269a3738ce5b2'),
    codigo: 'PROD-003',
    nombre: 'Lapiz',
    precio: 500,
    existencias: 15
  }
]
```



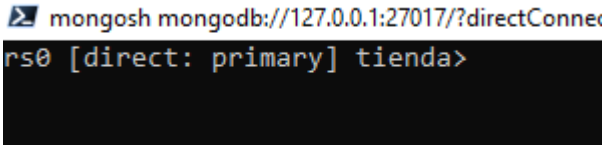


c. ¿Los identificadores _id de documentos insertados en el nodo secundario son los mismos que en el nodo primario?

Sí, los identificadores de los documentos “productos” insertados son los mismos que en el nodo primario, se insertaron igual para los nodos secundarios.

Prueba de fallo y recuperación de nodo

1. Establezca una conexión con alguno de los nodos del replica set usando el mongosh.



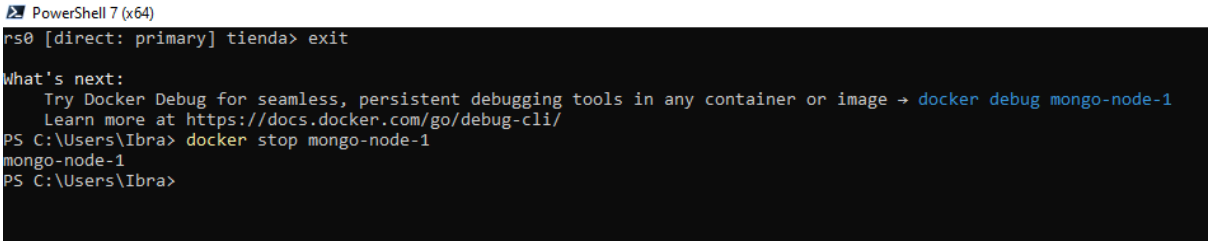
2. Ejecute el comando “rs.status()” en el nodo del replica set seleccionado usando el mongosh para verificar el status del replica set.

```
rs0 [direct: primary] tienda> rs.status()
{
  set: 'rs0',
  date: ISODate('2026-11-03T04:55:42.805Z'),
  myState: 1,
  term: Long('1'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1793681741, i: 1 }), t: Long('1') },
    lastCommittedWallTime: ISODate('2026-11-03T04:55:41.153Z'),
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1793681741, i: 1 }), t: Long('1') },
    appliedOpTime: { ts: Timestamp({ t: 1793681741, i: 1 }), t: Long('1') },
    durableOpTime: { ts: Timestamp({ t: 1793681741, i: 1 }), t: Long('1') },
    writtenOpTime: { ts: Timestamp({ t: 1793681741, i: 1 }), t: Long('1') },
    lastAppliedWallTime: ISODate('2026-11-03T04:55:41.153Z'),
    lastDurableWallTime: ISODate('2026-11-03T04:55:41.153Z'),
    lastWrittenWallTime: ISODate('2026-11-03T04:55:41.153Z')
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1793681741, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'electionTimeout',
    lastElectionDate: ISODate('2026-11-03T04:02:51.155Z'),
    electionTerm: Long('1'),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1793678561, i: 1 }), t: Long('-1') },
    lastSeenWrittenOpTimeAtElection: { ts: Timestamp({ t: 1793678561, i: 1 }), t: Long('-1') },
    lastSeenOpTimeAtElection: { ts: Timestamp({ t: 1793678561, i: 1 }), t: Long('-1') },
    numVotesNeeded: 2,
    priorityAtElection: 1,
    electionTimeoutMillis: Long('10000'),
    numCatchUpOps: Long('0'),
    newTermStartDate: ISODate('2026-11-03T04:02:51.210Z'),
    wMajorityWriteAvailabilityDate: ISODate('2026-11-03T04:02:51.691Z')
  },
  members: [
    {
      _id: 0,
      name: 'mongo-node-1:27017',
      health: 1
```

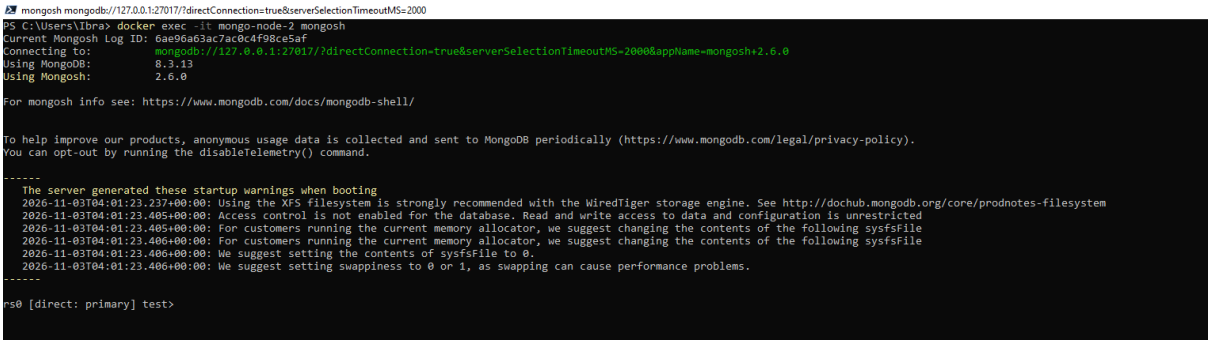
3. Tome una captura de pantalla donde se pueda identificar el nodo primario (“PRIMARY”) y los nodos secundarios (“SECONDARY”) y sus atributos “health”.

```
members: [
  {
    _id: 0,
    name: 'mongo-node-1:27017',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 3272,
    optime: { ts: Timestamp({ t: 1793681741, i: 1 }), t: Long('1') },
    optimeDate: ISODate('2026-11-03T04:55:41.000Z'),
    optimeWritten: { ts: Timestamp({ t: 1793681741, i: 1 }), t: Long('1') },
    optimeWrittenDate: ISODate('2026-11-03T04:55:41.000Z'),
    lastAppliedWallTime: ISODate('2026-11-03T04:55:41.153Z'),
    lastDurableWallTime: ISODate('2026-11-03T04:55:41.153Z'),
    lastWrittenWallTime: ISODate('2026-11-03T04:55:41.153Z'),
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1793678571, i: 1 }),
    electionDate: ISODate('2026-11-03T04:02:51.000Z'),
    configVersion: 1,
    configTerm: 1,
    self: true,
    lastHeartbeatMessage: ''
  },_id: 2,
  { name: 'mongo-node-3:27017',
    _id: 1, 1,
    name: 'mongo-node-2:27017',
    health: 1,'SECONDARY',
    state: 2,52,
    stateStr: 'SECONDARY',p({ t: 1793678811, i: 1 }), t: Long('1') },
    uptime: 3181,
    optime: { ts: Timestamp({ t: 1793681741, i: 1 }), t: Long('1') },
    optimeDurable: { ts: Timestamp({ t: 1793681741, i: 1 }), t: Long('1') },
    optimeWritten: { ts: Timestamp({ t: 1793681741, i: 1 }), t: Long('1') },
    optimeDate: ISODate('2026-11-03T04:55:41.000Z'),
    optimeDurableDate: ISODate('2026-11-03T04:55:41.000Z'),
    optimeWrittenDate: ISODate('2026-11-03T04:55:41.000Z'),
    lastAppliedWallTime: ISODate('2026-11-03T04:55:41.153Z'),
    lastDurableWallTime: ISODate('2026-11-03T04:55:41.153Z'),
    lastWrittenWallTime: ISODate('2026-11-03T04:55:41.153Z'),
    lastHeartbeat: ISODate('2026-11-03T04:55:41.186Z'),
    lastHeartbeatRecv: ISODate('2026-11-03T04:55:42.200Z'),
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
    syncSourceHost: 'mongo-node-1:27017',
    syncSourceId: 0,
    infoMessage: '',
    configVersion: 1,
    configTerm: 1
  },
  {
    _id: 2,
    name: 'mongo-node-3:27017',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 3181,
    optime: { ts: Timestamp({ t: 1793681741, i: 1 }), t: Long('1') },
    optimeDurable: { ts: Timestamp({ t: 1793681741, i: 1 }), t: Long('1') },
    optimeWritten: { ts: Timestamp({ t: 1793681741, i: 1 }), t: Long('1') },
    optimeDate: ISODate('2026-11-03T04:55:41.000Z'),
```

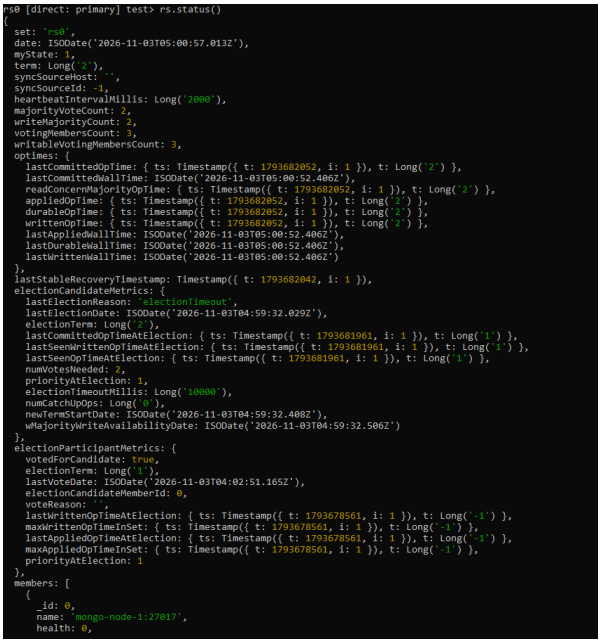
4. Detenga el contenedor que corresponde con el nodo primario identificado, utilizando sólo uno de los siguientes comandos según corresponda:



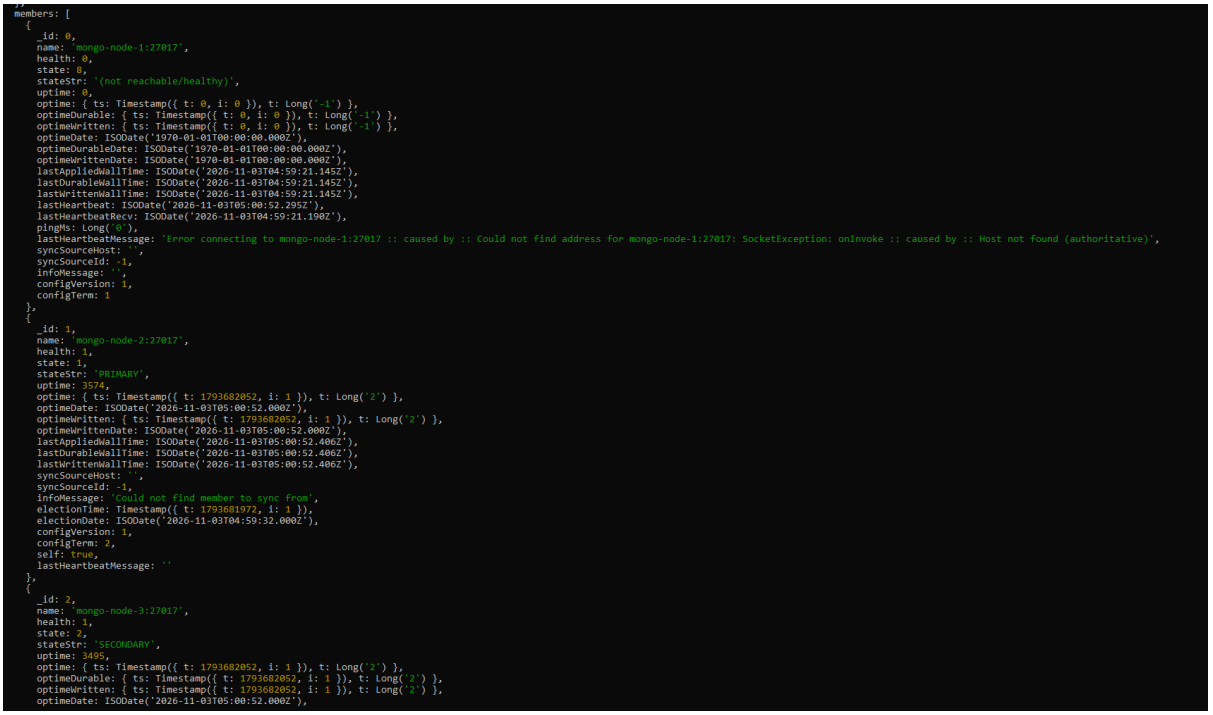
5. Establezca una conexión con alguno de los otros nodos del replica set que siguen activos usando el mongosh.



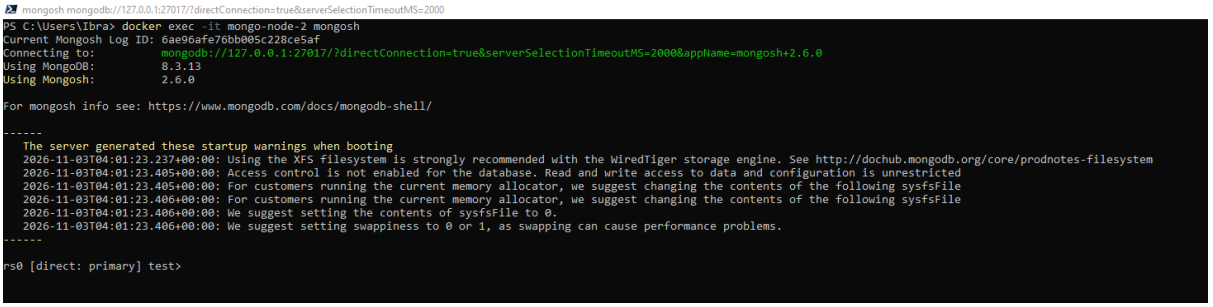
6. Ejecute el comando “rs.status()” en el nodo del replica set seleccionado usando el mongosh para verificar el status del replica set.



7. Tome una captura de pantalla donde se pueda identificar el nodo primario (“PRIMARY”) y los nodos secundarios (“SECONDARY”) y sus atributos “health”.



8. Establezca una conexión con el nuevo nodo primario del replica set usando el mongosh.



9. Ejecute los siguientes comandos para la creación de la base de datos y la colección.

```
rs0 [direct: primary] test> use tienda
switched to db tienda
rs0 [direct: primary] tienda>

rs0 [direct: primary] tienda> db.productos.insertOne({
...     codigo: "PROD-004",
...     nombre: "Borrador",
...     precio: 250,
...     existencias: 10
... })
{
  acknowledged: true,
  insertedId: ObjectId('6ae96b1e76bb005c228ce5b0')
}
rs0 [direct: primary] tienda>

rs0 [direct: primary] tienda> db.productos.find()
[
  {
    _id: ObjectId('6ae9659d9ee269a3738ce5b0'),
    codigo: 'PROD-001',
    nombre: 'Libro',
    precio: 25000,
    existencias: 5
  },
  {
    _id: ObjectId('6ae9659d9ee269a3738ce5b1'),
    codigo: 'PROD-002',
    nombre: 'Cuaderno ',
    precio: 3500,
    existencias: 25
  },
  {
    _id: ObjectId('6ae9659d9ee269a3738ce5b2'),
    codigo: 'PROD-003',
    nombre: 'Lapiz',
    precio: 500,
    existencias: 15
  },
  {
    _id: ObjectId('6ae96b1e76bb005c228ce5b0'),
    codigo: 'PROD-004',
    nombre: 'Borrador',
    precio: 250,
    existencias: 10
  }
]
rs0 [direct: primary] tienda>
```

10. Tome una captura de pantalla donde se observe el nuevo documento del producto insertado y se vea su identificador _id.

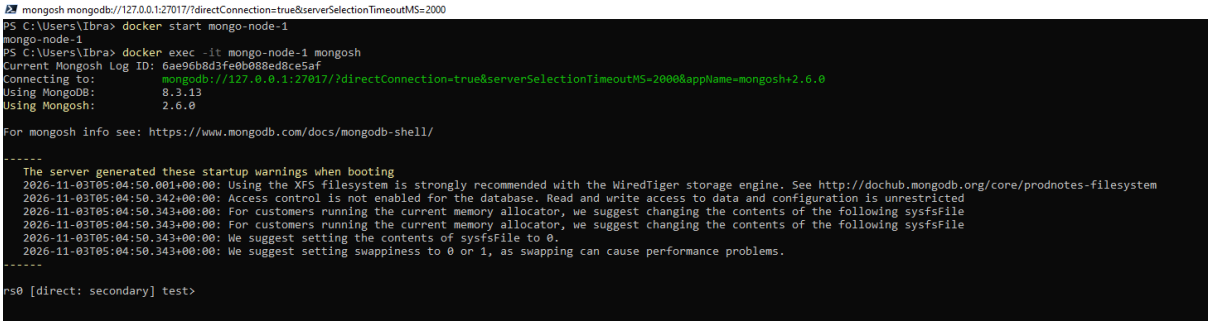
```
rs0 [direct: primary] tienda> db.productos.find()
[
  {
    _id: ObjectId('6ae9659d9ee269a3738ce5b0'),
    codigo: 'PROD-001',
    nombre: 'Libro',
    precio: 25000,
    existencias: 5
  },
  {
    _id: ObjectId('6ae9659d9ee269a3738ce5b1'),
    codigo: 'PROD-002',
    nombre: 'Cuaderno ',
    precio: 3500,
    existencias: 25
  },
  {
    _id: ObjectId('6ae9659d9ee269a3738ce5b2'),
    codigo: 'PROD-003',
    nombre: 'Lapiz',
    precio: 500,
    existencias: 15
  },
  {
    _id: ObjectId('6ae96b1e76bb005c228ce5b0'),
    codigo: 'PROD-004',
    nombre: 'Borrador',
    precio: 250,
    existencias: 10
  }
]
rs0 [direct: primary] tienda>
```

11. Inicie el contenedor que corresponde con el antiguo nodo primario identificado, utilizando sólo uno de los siguientes comandos según corresponda:

PowerShell 7 (x64)

```
PS C:\Users\Ibra> docker start mongo-node-1
mongo-node-1
PS C:\Users\Ibra>
```

12. Establezca una conexión con el antiguo nodo primario identificado del replica set usando el mongosh.



13. Ejecute los siguientes comandos para la verificación de la base de datos y la colección.



14. Ejecute el comando “rs.status()” para verificar el status del replica set.

```
members: [
  {
    _id: 0,
    name: 'mongo-node-1:27017',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 223,
    optime: { ts: Timestamp({ t: 1793682512, i: 1 }), t: Long('2') },
    optimeDate: ISODate('2026-11-03T05:08:32.000Z'),
    optimeWritten: { ts: Timestamp({ t: 1793682512, i: 1 }), t: Long('2') },
    optimeWrittenDate: ISODate('2026-11-03T05:08:32.000Z'),
    lastAppliedWallTime: ISODate('2026-11-03T05:08:32.392Z'),
    lastDurableWallTime: ISODate('2026-11-03T05:08:32.392Z'),
    lastWrittenWallTime: ISODate('2026-11-03T05:08:32.392Z'),
    syncSourceHost: 'mongo-node-2:27017',
    syncSourceId: 1,
    infoMessage: '',
    configVersion: 1,
    configTerm: 2,
    self: true,
    lastHeartbeatMessage: ''
  },
  {
    _id: 1,
    name: 'mongo-node-2:27017',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 222,
    optime: { ts: Timestamp({ t: 1793682502, i: 1 }), t: Long('2') },
    optimeDurable: { ts: Timestamp({ t: 1793682502, i: 1 }), t: Long('2') },
    optimeWritten: { ts: Timestamp({ t: 1793682502, i: 1 }), t: Long('2') },
    optimeDate: ISODate('2026-11-03T05:08:22.000Z'),
    optimeDurableDate: ISODate('2026-11-03T05:08:22.000Z'),
    optimeWrittenDate: ISODate('2026-11-03T05:08:22.000Z'),
    lastAppliedWallTime: ISODate('2026-11-03T05:08:22.392Z'),
    lastDurableWallTime: ISODate('2026-11-03T05:08:22.392Z'),
    lastWrittenWallTime: ISODate('2026-11-03T05:08:22.392Z'),
    lastHeartbeat: ISODate('2026-11-03T05:08:31.893Z'),
    lastHeartbeatRecv: ISODate('2026-11-03T05:08:32.371Z'),
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1793681972, i: 1 }),
    electionDate: ISODate('2026-11-03T04:59:32.000Z'),
    configVersion: 1,
    configTerm: 2
  },
  {
    _id: 2,
    name: 'mongo-node-3:27017',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 222,
    optime: { ts: Timestamp({ t: 1793682502, i: 1 }), t: Long('2') },
    optimeDurable: { ts: Timestamp({ t: 1793682502, i: 1 }), t: Long('2') },
    optimeWritten: { ts: Timestamp({ t: 1793682502, i: 1 }), t: Long('2') },
    optimeDate: ISODate('2026-11-03T05:08:22.000Z'),
```

15. Conteste las siguientes preguntas

a. ¿El nodo reiniciado volvió como `PRIMARY` o como `SECONDARY`?

En esta ocasión, el nodo reiniciado (que antes era Primary) volvió como Secondary.

b. ¿El documento PROD-004, insertado mientras ese nodo estaba detenido, está actualmente en ese nodo?

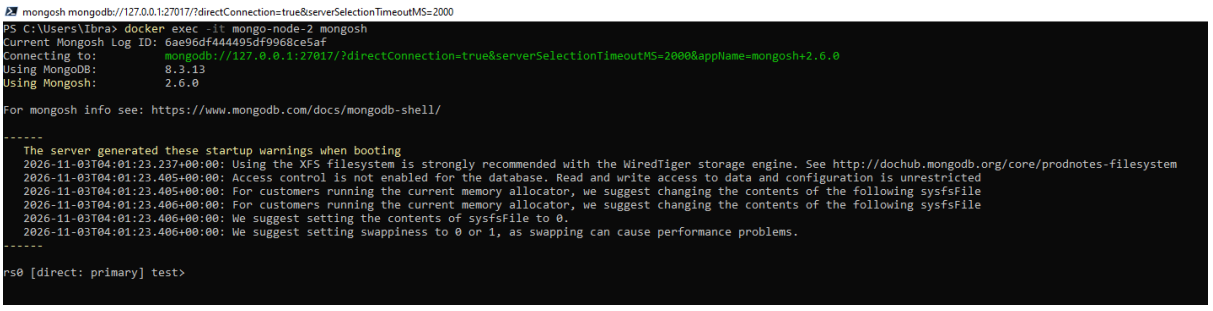
Sí, luego de reiniciar el nodo y consultar los documentos, se encontró el documento PROD-004 en el nodo.

c. ¿Qué demuestra esto sobre la recuperación y sincronización de un miembro del replica set?

Esto demuestra que un miembro del replica set que se cae puede reincorporarse solo y sincronizarse automáricamente. Al reiniciar el nodo 1, volvió como SECONDARY y tenía el documento PROD-004, insertado mientras estaba detenido, porque aplicó las operaciones que se perdió leyendo el log del nuevo primario.

Prueba de transacciones

1. Establezca una conexión con el nodo primario del replica set usando el mongosh, puede ejecutar el comando “rs.status()” en uno de los nodos del replica set para identificar primario.



2. Ejecute los siguientes comandos para la establecer los datos que se utilizaran en la prueba.

```
mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=
rs0 [direct: primary] test> use tienda
switched to db tienda
rs0 [direct: primary] tienda>

rs0 [direct: primary] tienda> db.productos.insertOne({
...     codigo: "PROD-005",
...     nombre: "Lapicero",
...     precio: 750,
...     existencias: 20
... })
{
  acknowledged: true,
  insertedId: ObjectId('6ae96e9644495df9968ce5b0')
}
rs0 [direct: primary] tienda>

rs0 [direct: primary] tienda> db.productos.find()
[
  {
    _id: ObjectId('6ae9659d9ee269a3738ce5b0'),
    codigo: 'PROD-001',
    nombre: 'Libro',
    precio: 25000,
    existencias: 5
  },
  {
    _id: ObjectId('6ae9659d9ee269a3738ce5b1'),
    codigo: 'PROD-002',
    nombre: 'Cuaderno ',
    precio: 3500,
    existencias: 25
  },
  {
    _id: ObjectId('6ae9659d9ee269a3738ce5b2'),
    codigo: 'PROD-003',
    nombre: 'Lapiz',
    precio: 500,
    existencias: 15
  },
  {
    _id: ObjectId('6ae96b1e76bb005c228ce5b0'),
    codigo: 'PROD-004',
    nombre: 'Borrador',
    precio: 250,
    existencias: 10
  },
  {
    _id: ObjectId('6ae96e9644495df9968ce5b0'),
    codigo: 'PROD-005',
    nombre: 'Lapicero',
    precio: 750,
    existencias: 20
  }
]
rs0 [direct: primary] tienda>
```

3. Ejecute los siguientes comandos para realizar una transacción exitosa. La transacción inserta una orden y descuenta los productos del inventario.

```
mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
rs0 [direct: primary] tienda> const session = db.getMongo().startSession()

rs0 [direct: primary] tienda> try {
...     session.startTransaction()
...
...     const tienda = session.getDatabase("tienda")
...
...     tienda.ordenes.insertOne({
...         numero: "ORD-001",
...         estado: "pendiente",
...         items: [
...             {
...                 codigoProducto: "PROD-005",
...                 nombre: "Lapicero",
...                 precio: 750,
...                 cantidad: 10
...             }
...         ],
...         total: 7500
...     })
...
...     tienda.productos.updateOne(
...         { codigo: "PROD-005" },
...         { $inc: { existencias: -10 } }
...     )
...
...     session.commitTransaction()
...
...     print("Transacción comprometida")
... } catch (error) {
...     session.abortTransaction()
...
...     print("Transacción abortada: " + error)
... } finally {
...     session.endSession()
... }
Transacción comprometida

rs0 [direct: primary] tienda>
```

4. Tome una captura de pantalla donde se observe los datos de la orden y del producto, luego de ejecutar la transacción exitosa.

```
mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
rs0 [direct: primary] tienda> db.ordenes.find({ numero: "ORD-001" })
[
  {
    _id: ObjectId('6ae96f1044495df9968ce5b1'),
    numero: 'ORD-001',
    estado: 'pendiente',
    items: [
      {
        codigoProducto: 'PROD-005',
        nombre: 'Lapicero',
        precio: 750,
        cantidad: 10
      }
    ],
    total: 7500
  }
]
rs0 [direct: primary] tienda> db.productos.find({ codigo: "PROD-005" })
[
  {
    _id: ObjectId('6ae96e9644495df9968ce5b0'),
    codigo: 'PROD-005',
    nombre: 'Lapicero',
    precio: 750,
    existencias: 10
  }
]
rs0 [direct: primary] tienda>
```

5. Ejecute los siguientes comandos para realizar una transacción fallida. La transacción inserta una orden y descuenta los productos del inventario, pero genera un error previo a comprometer la transacción.

```
mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
rs0 [direct: primary] tienda> const session = db.getMongo().startSession()

rs0 [direct: primary] tienda> try {
...     session.startTransaction()
...
...     const tienda = session.getDatabase("tienda")
...
...     tienda.ordenes.insertOne({
...         numero: "ORD-001",
...         estado: "pendiente",
...         items: [
...             {
...                 codigoProducto: "PROD-005",
...                 nombre: "Lapicero",
...                 precio: 750,
...                 cantidad: 10
...             }
...         ],
...         total: 7500
...     })
...
...     tienda.productos.updateOne(
...         { codigo: "PROD-005" },
...         { $inc: { existencias: -10 } }
...     )
...
...     throw new Error("Error intencional")
...
...     session.commitTransaction()
...
...     print("Transacción comprometida")
... } catch (error) {
...     session.abortTransaction()
...
...     print("Transacción abortada: " + error)
... } finally {
...     session.endSession()
... }
Transacción abortada: Error: Error intencional

rs0 [direct: primary] tienda>
```

6. Tome una captura de pantalla donde se observe los datos de la orden y del producto, luego de ejecutar la transacción fallida.

```
mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
rs0 [direct: primary] tienda> db.ordenes.find({ numero: "ORD-001" })
[
  {
    _id: ObjectId('6ae96f1044495df9968ce5b1'),
    numero: 'ORD-001',
    estado: 'pendiente',
    items: [
      {
        codigoProducto: 'PROD-005',
        nombre: 'Lapicero',
        precio: 750,
        cantidad: 10
      }
    ],
    total: 7500
  }
]
rs0 [direct: primary] tienda> db.productos.find({ codigo: "PROD-005" })
[
  {
    _id: ObjectId('6ae96e9644495df9968ce5b0'),
    codigo: 'PROD-005',
    nombre: 'Lapicero',
    precio: 750,
    existencias: 10
  }
]
rs0 [direct: primary] tienda>
```

Eliminación del Replica Set

1. Ejecute los siguientes comandos para eliminar los contenedores y la red interna del replica set de Mongo. Asegúrese de haber terminado esta parte del laboratorio, ya que la eliminación no se puede deshacer.

```
PowerShell 7 (x64)
PS C:\Users\Ibra> docker rm -f mongo-node-1
mongo-node-1
PS C:\Users\Ibra> docker rm -f mongo-node-2
mongo-node-2
PS C:\Users\Ibra> docker rm -f mongo-node-3
mongo-node-3
PS C:\Users\Ibra> docker network rm mongo-net
mongo-net
PS C:\Users\Ibra>
```